



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

**Secure CompilerAI: A Semantic Compiler for Intelligent Vulnerability
Detection and Verified Code Optimization**

A CAPSTONE PROJECT REPORT

Submitted in the partial fulfilment for the Course of

CSA 1403-COMPILER DESIGN

to the award of the degree of

BACHELOR OF ENGINEERING

IN

B.Tech-IT

B.E-CSE

B.Tech-IT

Submitted by

POOJA SREE M (192521403)

SOFIA R V (192511055)

PRARTANA T (192421099)

Under the Supervision of Dr.

G.Michael

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

September 2026

SIMATS ENGINEERING

**Saveetha Institute of Medical and Technical
Sciences Chennai-602105**

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled “**Secure CompilerAI: A Semantic Compiler for Intelligent Vulnerability Detection and Verified Code Optimization**” has been carried out by **Pooja Sree M**, Sofia R V, Prartana T under the supervision of **Dr. G. Michael** and is submitted in partial fulfilment of the requirements for the current semester of the **B.Tech-IT, B.E-CSE, B.Tech-IT** program at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR

Dr.W.Deva Priya,
Professor
Computer science and engineering
SIMATS Engineering
Chennai – 602105.

COURSE FACULTY

Dr. G. MICHAEL
Professor
Computer science and engineering
SIMATS Engineering
Chennai – 602105.

SIMATS ENGINEERING

**Saveetha Institute of Medical and Technical
Sciences Chennai-602105**

DECLARATION

We, by **Pooja Sree M**, Sofia R V, Prartana T of the **B.Tech-Information Technology, B.E-Computer science and engineering, B.Tech-Information Technology**, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled '**Secure CompilerAI: A Semantic Compiler for Intelligent Vulnerability Detection and Verified Code Optimization**' is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place: Chennai

Date:

Signature of the Students with Names

Pooja Sree M(192521403)

SOFIA R V (192511055)

PRARTANA T (192421099)

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

Individual Contribution Statement

(Mandatory for all team projects)

Student Name / Register No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. Contribution (%)
Pooja Sree M (192521403)	Module 1 – Intelligent Source Code Analysis & Secure Parsing; source-code representation and secure parsing workflow.	Contributed to lexical analysis, secure parsing, AST/symbol-table representation, and integration of the front-end analysis flow.	Tested valid/invalid syntax, parsing behaviour, structural representations, and source-analysis cases.	Introduction, Module 1 description, methodology, testing approach, and related analysis.	34%
Sofia R V (192511055)	Module 2 – Semantic Security Verification Engine; security-aware semantic analysis.	Contributed to control-flow/data-flow reasoning, vulnerability identification, security constraints, and verification conditions.	Tested vulnerability-analysis cases, source-to-sink reasoning, security checks, and negative cases.	Literature review, Module 2 description, verification/testing discussion, and analysis.	33%
Prartana T (192421099)	Module 3 – Verified Secure Code Transformation & Explainable Optimization.	Contributed to candidate repair generation, transformation, re-analysis, semantic-preservation checks, and explainable output.	Tested accepted/rejected transformations, regression behaviour, and transformation verification.	Module 3, architecture, results discussion, conclusion, future work, and supporting documentation.	33%
Total					100%

ABSTRACT

Secure CompilerAI is a semantic-aware compiler framework designed to improve software security by integrating intelligent source-code analysis, semantic verification, verified secure code transformation, and explainable security optimization into the compilation workflow. Traditional compiler pipelines primarily focus on lexical, syntactic, and semantic correctness, while security analysis is often performed as a separate static-analysis or review activity. This separation can allow security weaknesses to persist until late in the development process.

The proposed Secure CompilerAI architecture introduces three integrated modules. Module 1, Intelligent Source Code Analysis & Secure Parsing, performs lexical and syntactic analysis while constructing security-aware representations of source programs. Module 2, the Semantic Security Verification Engine, examines data flow, control flow, program states, and security properties to identify potential vulnerabilities and validate security constraints. Module 3, Verified Secure Code Transformation & Explainable Optimization, generates security-preserving transformations and provides human-readable explanations for why a transformation is safe and beneficial.

The framework is intended to combine compiler theory with AI-assisted security reasoning. Its design emphasizes correctness, traceability, explainability, and verification so that automated changes do not merely reduce warnings but preserve program semantics. The project therefore demonstrates how compiler phases can be extended with security intelligence while maintaining a structured compilation pipeline. The approach is suitable for educational compiler laboratories, secure software-development workflows, and future research into trustworthy AI-assisted program transformation.

TABLE OF CONTENTS

Sl. No	Title	Page No
i	CERTIFICATE FROM THE SUPERVISOR	ii
ii	DECLARATION BY THE CANDIDATE	iii
lii	INDIVIDUAL CONTRIBUTION STATEMENT	iv
iv	ABSTRACT	v
v	LIST OF FIGURES	vi
vi	LIST OF TABLES	vii
vii	LIST OF ABBREVIATIONS	viii
1	CHAPTER 1: Introduction	1
2	CHAPTER 2: Literature Review	10
3	CHAPTER 3: Engineering Design & Trade-offs, Sustainability, Ethics, Safety, Risk & Security	20
4	CHAPTER 4: Implementation, Testing & Results, Project Management	34
5	CHAPTER 5: Conclusion & Reflection	38
	References	39
	Appendices	40

LIST OF FIGURES

Figure No.	Figure Name	Description
Figure 1	Development and Design Process	Overall project development sequence
Figure 2	Module 1 – Intelligent Source Code Analysis & Secure Parsing	Front-end parsing and structured representation
Figure 3	Module 2 – Semantic Security Verification Engine	Security-aware semantic analysis and verification
Figure 4	Module 3 – Verified Secure Code Transformation & Explainable Optimization	Verified transformation and explanation
Figure 5	System Architecture	End-to-end Secure CompilerAI pipeline

TABLE OF CONTENTS

Sl. No.	Title
1	CHAPTER 1 – INTRODUCTION AND ENGINEERING PROBLEM
1.1	Background
1.2	Need for the Project
1.3	Problem Statement
1.4	Project Aim
1.5	Project Objectives
1.6	Scope
1.7	Expected Engineering Outcome
2	CHAPTER 2 – LITERATURE REVIEW AND EXISTING SOLUTIONS
2.1	Review Method
2.2	Review of Existing Work
2.3	Comparative Analysis
2.4	Research / Engineering Gap
2.5	Proposed Contribution
3	CHAPTER 3 – ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY
3.1	Requirements
3.2	Constraints & Applicable Standards
3.3	Design Specifications
3.4	Alternative Solutions & Evaluation
3.5	Selected Design & Detailed Design
3.6	Trade-off Analysis
3.7	Sustainability, Ethics, Safety, Risk & Security
4	CHAPTER 4 – IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT
4.1	Development Approach & Resources
4.2	Software Implementation & Modern Tools
4.3	Test Plan & Verification

4.4	Results
4.5	Data Analysis & Engineering Judgment
4.6	Comparison with Existing Solutions & Achievement of Objectives
4.7	Strengths & Limitations
4.8	Project Planning, Roles & Budget
5	CHAPTER 5 – CONCLUSION, FUTURE WORK AND REFLECTION
5.1	Conclusion
5.2	Future Work
5.3	Professional Learning & Individual Reflection
	REFERENCES
	APPENDICES A–J

LIST OF FIGURES

Figure No.	Figure Name	Description
Figure 1	Development and Design Process	Overall project development sequence
Figure 2	Module 1 – Intelligent Source Code Analysis & Secure Parsing	Front-end parsing and structured representation
Figure 3	Module 2 – Semantic Security Verification Engine	Security-aware semantic analysis and verification
Figure 4	Module 3 – Verified Secure Code Transformation & Explainable Optimization	Verified transformation and explanation
Figure 5	System Architecture	End-to-end Secure CompilerAI pipeline

LIST OF TABLES

Table No.	Table Name
Table 1	Individual Contribution Statement
Table 2	Stakeholder / User Requirements
Table 3	Functional and Non-Functional Requirements
Table 4	Constraints and Applicable Standards
Table 5	Design Specifications
Table 6	Alternative Solution Evaluation
Table 7	Trade-off Analysis
Table 8	Sustainability, Ethics, Safety and Security
Table 9	Risk Register
Table 10	Software and Tools Used
Table 11	Test Plan and Verification
Table 12	Achievement of Objectives
Table 13	Project Roles

LIST OF ABBREVIATIONS / SYMBOLS

Abbreviation / Symbol	Description
AI	Artificial Intelligence
AST	Abstract Syntax Tree
CFG	Control Flow Graph
DFG	Data Flow Graph
IR	Intermediate Representation
LLM	Large Language Model
SMT	Satisfiability Modulo Theories
CWE	Common Weakness Enumeration
CVE	Common Vulnerabilities and Exposures
CI/CD	Continuous Integration / Continuous Delivery
API	Application Programming Interface

CHAPTER 1 – INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

Compiler design traditionally concerns the conversion of source programs into executable or intermediate representations through lexical analysis, syntax analysis, semantic analysis, intermediate-code generation, optimization, and target-code generation. Modern software systems must additionally satisfy security requirements. Vulnerabilities can originate from unsafe input handling, incorrect memory operations, injection-prone data flows, insecure API usage, and other patterns that may remain syntactically and semantically valid.

Secure CompilerAI extends the compiler perspective by treating security as a property that should be considered during program analysis and transformation. The system combines secure parsing, semantic security verification, and verified transformation into a single conceptual pipeline. AI-assisted analysis can help identify suspicious patterns and prioritize security concerns, while compiler-based verification provides structural and semantic evidence before an automated change is accepted.

1.2 Need for the Project

The need for Secure CompilerAI arises from the gap between traditional compilation and security-oriented development. Conventional compilers can establish that a program follows language rules without necessarily determining whether the program is secure against misuse, untrusted input, unsafe data flow, or insecure API usage. Separate security tools can provide useful warnings, but the developer may still need to interpret the finding, design a repair, and confirm that the repair does not change intended behaviour.

- Early detection of security weaknesses can reduce the cost and effort associated with late-stage remediation.
- Semantic information provides more context than surface-level pattern matching when the same syntax can be safe in one situation and unsafe in another.
- Automated fixes require verification because a generated change may introduce functional regressions or fail to address the actual security property.
- Explainable findings improve developer understanding by connecting the reported issue to evidence, proposed changes, and verification status.

1.3 Problem Statement

The engineering problem is to design a compiler-oriented security framework that can analyse source programs using syntactic and semantic information, identify security-relevant behaviour, propose secure transformations, and verify the transformations before acceptance. The problem contains interacting requirements: security improvement, semantic correctness, performance, explainability, maintainability, and extensibility must be considered together.

A purely pattern-based detector may miss contextual vulnerabilities or generate false positives. Conversely, an unrestricted AI-based repair mechanism may generate code that appears secure but changes program behaviour. Therefore, the proposed solution requires a controlled interaction between AI assistance and deterministic compiler/security verification.

1.4 Project Aim

To design and document a semantic-aware compiler framework that integrates intelligent vulnerability detection, security verification, verified code transformation, and explainable optimization into a structured compilation workflow.

1.5 Project Objectives

1. Design a semantic-aware compiler framework that integrates security analysis into compilation.
2. Develop intelligent source-code analysis and secure parsing mechanisms for structured program representations.
3. Identify potential vulnerabilities using semantic, control-flow, and data-flow information.
4. Verify security properties before and after code transformations.
5. Generate secure code transformations that preserve intended program behaviour.
6. Provide explainable optimization results describing detected risks, applied changes, and verification evidence.
7. Create a modular architecture that can be extended to multiple programming languages and security rules.

1.6 Scope

Included scope: secure lexical and syntactic processing, intermediate representation, security-aware semantic analysis, vulnerability identification, verification rules, secure code transformation, and explainable optimization. The project focuses on a prototype architecture demonstrating interaction among these phases.

Excluded scope: production-grade support for every programming language, complete formal verification of all possible program properties, and replacement of expert security review. The AI component is treated as an assistant whose recommendations require compiler-level validation rather than as an unrestricted autonomous code generator.

Assumptions include availability of a structured source representation, defined security rules, a supported programming language grammar, and a verification mechanism appropriate to the transformation being considered. The principal boundary condition is that a candidate transformation is accepted only when the defined security and semantic checks are satisfied.

1.7 Expected Engineering Outcome

The intended outcome is a modular compiler-security prototype architecture and associated engineering documentation. The system is expected to receive source code, construct structured representations such as AST, CFG, DFG and symbol tables, perform security-aware analysis, produce candidate remediation, re-analyse the transformed program, and present an explainable verification result.

CHAPTER 2 – LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

The review was organized around established compiler concepts, software-security analysis, program transformation, verification, and AI-assisted developer tooling. The source report identifies compiler textbooks, software-quality standards, vulnerability classifications, parser technology, compiler infrastructure, Python/AST documentation, refactoring concepts, and software-verification research as relevant foundations.

The review therefore focuses on the engineering roles of lexical analysis, parsing, AST/IR construction, data-flow and control-flow reasoning, vulnerability classification, secure coding, semantic-preservation checking, and explainable security feedback.

2.2 Review of Existing Work

Compiler Pipelines

Classical compiler approaches provide a well-defined sequence from source text to structured representations and eventually optimized or target code. Their major advantage is strong organization of syntax and semantics. However, security analysis is not inherently the primary objective of a traditional compiler pipeline.

Pattern-Based Security Analysis

Pattern and rule-based security detection is effective for recognizable coding weaknesses and provides deterministic behaviour. Its limitation is contextual understanding: the same syntax may be safe or unsafe depending on data flow, control conditions, sanitization, and program state.

Static Program Analysis

Static-analysis approaches use program structure, data flow, control flow, and semantic information to reason about potential weaknesses without executing every possible path. These techniques provide stronger context than isolated signatures, but analysis can become computationally expensive as program size and path complexity increase.

AI-Assisted Code Analysis and Transformation

AI-assisted reasoning can help prioritize findings, generate explanations, and propose candidate repairs. The project source emphasizes that AI should remain an assisting layer and should not bypass deterministic compiler and security checks. This is important because generated code may introduce regressions, alter intended behaviour, or fail to satisfy the underlying security property.

2.3 Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
Traditional compiler pipeline	Strong syntax/semantic processing; structured representations	Security is not necessarily integrated into compilation	Provides the compiler foundation
Pattern-based security rules	Deterministic and effective for known patterns	Context-dependent false positives/negatives	Used as part of security rules
Static analysis	Uses control/data-flow context	Can be complex and computationally expensive	Supports semantic security verification
AI-assisted code generation	Flexible recommendations and explanations	May introduce incorrect or insecure changes	Used only as candidate-generation assistance
Secure CompilerAI	Combines parsing, semantic security, verification and explainability	Prototype scope and language/rule limitations	Proposed integrated framework

2.4 Research / Engineering Gap

The unresolved engineering gap is the safe connection between vulnerability detection and automated remediation. Detection alone produces findings that still require developer interpretation. Automated remediation alone can be unsafe if the generated transformation is not checked for semantic preservation. Secure CompilerAI addresses this gap conceptually by placing re-analysis and verification between candidate transformation and acceptance.

2.5 Proposed Contribution

The proposed contribution is a three-module architecture in which each module has a defined responsibility. Module 1 constructs trustworthy program representations; Module 2 evaluates those representations using security-aware semantic reasoning; and Module 3 generates, verifies, and explains security-preserving transformations. The architecture is modular so that languages, security policies, analysis methods, and transformation rules can be extended independently.

CHAPTER 3 – ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

3.1.1 Stakeholder / User Requirements

Stakeholder	Requirement
Software Developers	Early, actionable security feedback and safe automated fixes.
Security Engineers	Precise findings, traceable evidence, and reduced false positives.
Compiler Designers	Security analysis and transformation mechanisms that fit compiler phases.
DevOps / Engineering Teams	Security checks that can integrate with development and CI/CD workflows.
Students / Researchers	A modular framework for studying compiler design, program analysis, and trustworthy AI.

3.1.2 Functional & Non-Functional Requirements

Requirement	Type	Measurable / Checkable Target
Tokenize source code	Functional	Source input produces structured tokens for supported syntax.
Parse source securely	Functional	Valid and invalid syntax are distinguished and AST is constructed for valid input.
Construct AST / IR / symbol information	Functional	Required program structures are available to later analysis.
Perform security-aware semantic analysis	Functional	Security-relevant data/control relationships are evaluated.
Generate vulnerability findings	Functional	Finding includes type, location, affected flow and supporting evidence.
Generate candidate transformation	Functional	Candidate repair is produced for supported vulnerability/transformation rules.

Verify transformed program	Functional	Security and semantic checks are re-run before acceptance.
Explain result	Functional	Output states risk, change, verification status and rationale.
Maintainability	Non-Functional	Modules and rules remain separable and traceable.
Reliability	Non-Functional	Regression tests are used for transformation changes.
Usability	Non-Functional	Security feedback is understandable to developers.

3.2 Constraints & Applicable Standards

Standard / Practice	Requirement / Purpose	How Applied in This Project
ISO/IEC 25010	Software quality characteristics	Used as a quality reference for functionality, reliability, usability, efficiency and maintainability.
IEEE-style testing practices	Organized test cases and verification evidence	Used to structure test inputs, expected outcomes and regression checks.
OWASP Top 10	Common web application security risks	Provides a security-awareness reference for vulnerability classes and secure coding discussion.
MITRE CWE	Software weakness classification	Provides a common vocabulary for describing software weaknesses.
MITRE CVE	Vulnerability identification reference	Provides a reference model for security-vulnerability identification.
Version control / traceability	Reproducible development history	Git is identified for change tracking and transformation histories.

3.3 Design Specifications

Parameter	Target Requirement /	Unit	Priority
Source representation	Structured token/AST/IR information available	representation	High
Semantic analysis	Control-flow and data-flow context available for supported cases	analysis	High
Vulnerability evidence	Finding includes location, severity/context and evidence	finding	High
Transformation safety	Re-analysis and verification before acceptance	verification	High
Explainability	Risk, change, expected effect and verification status reported	report	Medium
Extensibility	Modules/rules can be expanded without redesigning the complete pipeline	architecture	Medium

3.4 Alternative Solutions & Evaluation

Alternative 1 – Separate compiler and security scanner: the compiler remains unchanged while an independent security tool analyses source code. This is simple to integrate but can create duplicated processing and weaker linkage between findings and transformations.

Alternative 2 – AI-only code security assistant: an AI system identifies vulnerabilities and generates fixes directly. This offers flexibility but does not provide sufficient deterministic assurance by itself.

Alternative 3 – Integrated semantic compiler-security framework: compiler representations are reused for security analysis and every candidate transformation is re-analysed and verified. This is the selected architecture because it directly addresses the project’s central requirement of verification before acceptance.

Criterion	Weight	Alt. 1: Separate Scanner	Alt. 2: AI-only Assistant	Alt. 3: Integrated Framework
Security assurance	High	Medium	Low–Medium	High
Semantic context	High	Medium	Variable	High
Explainability	Medium	Medium	High	High
Transformation safety	High	Medium	Low	High
Extensibility	Medium	High	High	High
Integration complexity	Medium	Medium	Medium	Medium–High

3.5 Selected Design & Detailed Design

The selected design follows a sequential but modular pipeline: Source Code → Lexer → Secure Parser → AST/IR → Semantic Security Verification → Vulnerability Findings → AI-Assisted Recommendation → Candidate Transformation → Re-analysis & Verification → Explainable Optimized Code.

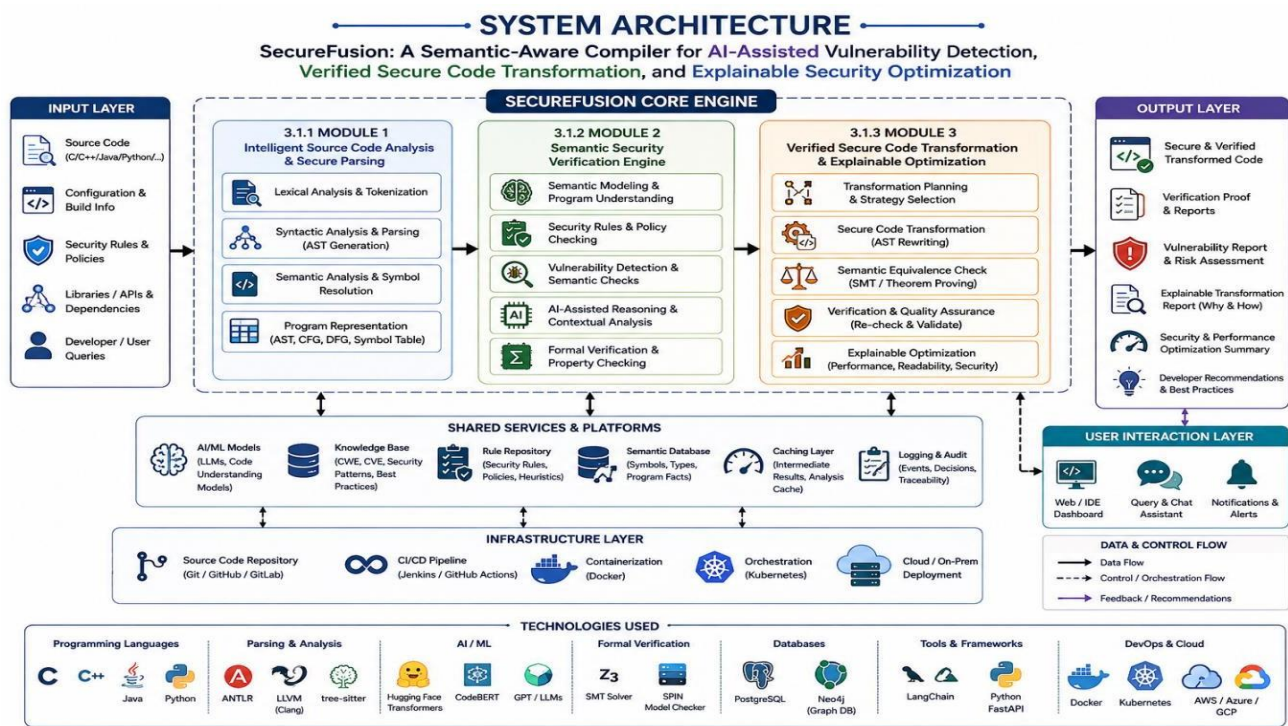


Figure 5. System Architecture of Secure CompilerAI

Module 1 – Intelligent Source Code Analysis & Secure Parsing

Module 1 forms the front end of Secure CompilerAI. It converts source code into structured compiler representations while preserving information needed for security reasoning. Lexical analysis identifies keywords, identifiers, literals, operators, delimiters and comments. Secure parsing validates grammatical structure and constructs an AST. Symbol-table construction records declarations, scopes, types and relevant program entities. Security metadata can be attached to nodes representing sources, sinks, sanitizers, sensitive operations or security-relevant APIs.

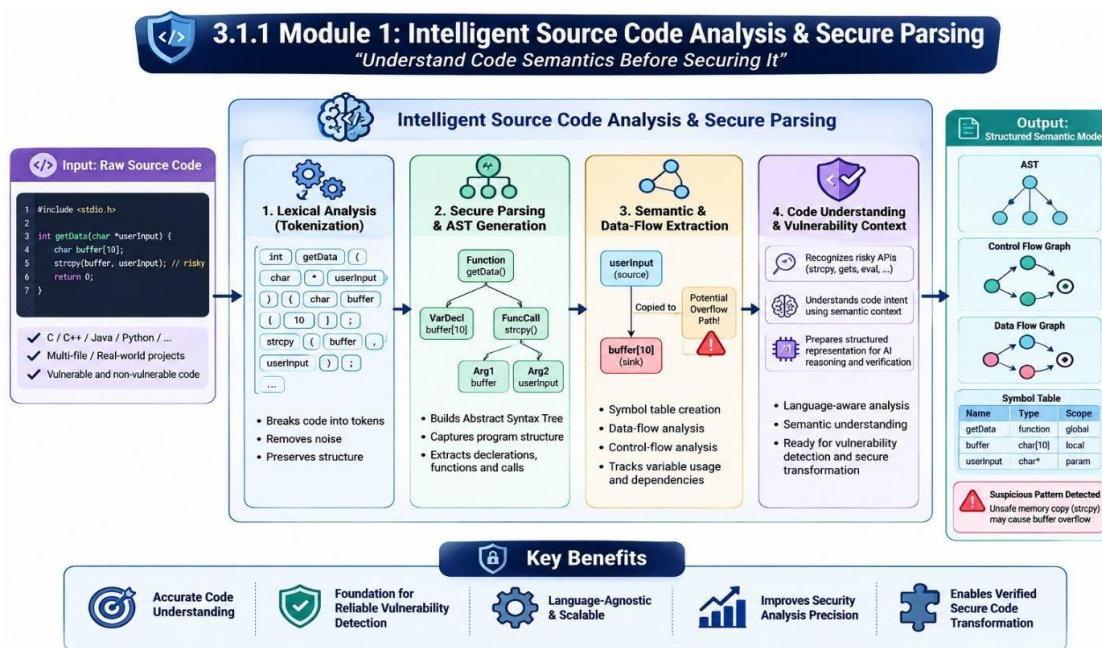


Figure 2. Module 1 – Intelligent Source Code Analysis & Secure Parsing

Module 2 – Semantic Security Verification Engine

Module 2 analyses the structured program using semantic information. It combines conventional compiler analysis with security rules to determine whether suspicious data or control flows violate defined security properties. The module builds control-flow and data-flow representations, tracks values from potentially untrusted sources to security-sensitive sinks, evaluates sanitization/validation/authorization/type constraints where applicable, and generates vulnerability findings with severity, location, affected flow and supporting evidence.

3.1.2 Module 2: Semantic Security Verification Engine

Verifies code semantics, detects vulnerabilities, and ensures security properties using formal methods and AI-assisted reasoning.

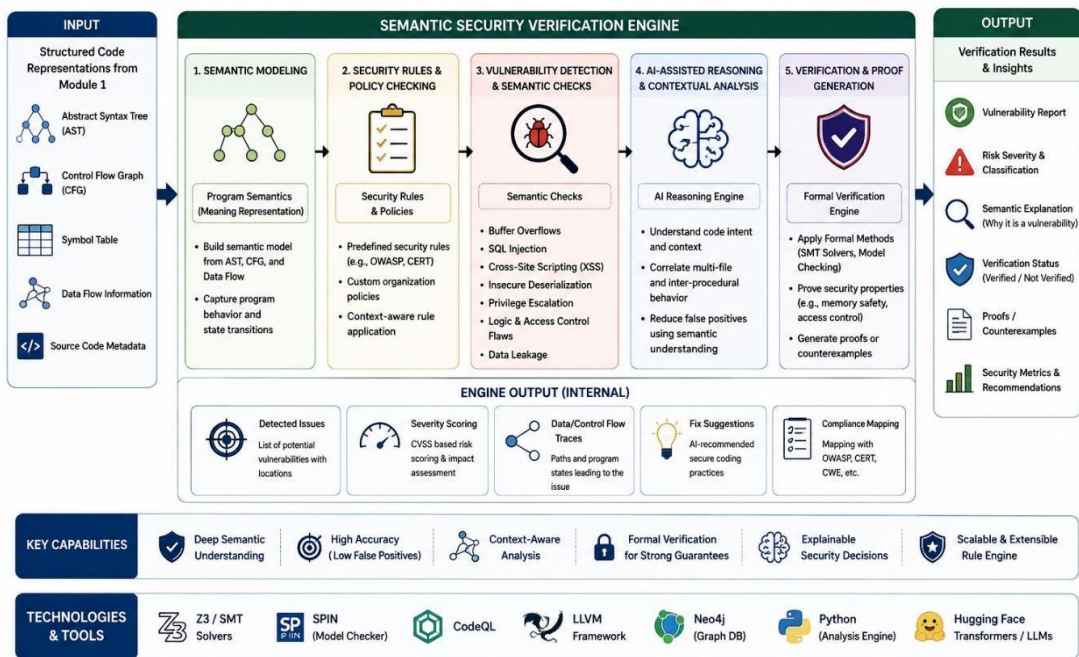


Figure 3. Module 2 – Semantic Security Verification Engine

Module 3 – Verified Secure Code Transformation & Explainable Optimization

Module 3 generates candidate repairs or optimizations and validates them before acceptance. Supported transformation concepts include input validation, safer API replacement, escaping or sanitization insertion, bounds checks, safer resource handling, and equivalent security-preserving rewrites depending on the language and rule set. The process selects a vulnerability, constructs a candidate, applies it to source or IR, re-runs semantic/security analysis, checks preservation conditions, accepts only verified transformations, and generates an explanation.

3.1.3 Module 3: Verified Secure Code Transformation & Explainable Optimization

Transforms vulnerable code into secure, semantically equivalent code with formal verification and provides clear explanations and optimization insights.

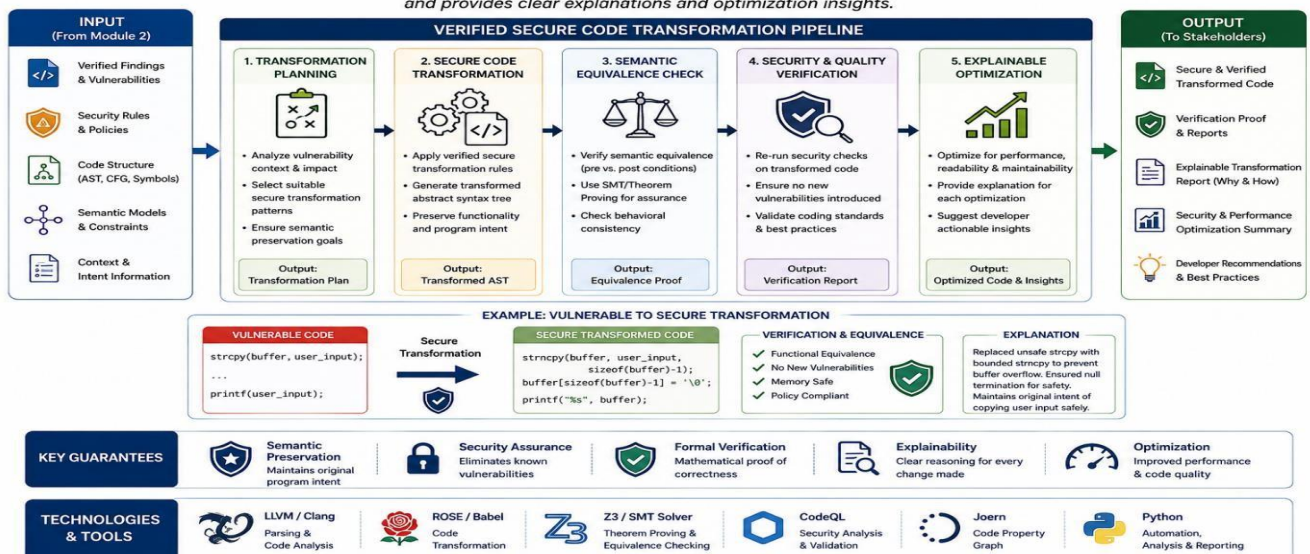


Figure 4. Module 3 – Verified Secure Code Transformation & Explainable Optimization

3.5.1 Systems Approach

The modules depend on one another but retain clear boundaries. Module 1 supplies the trustworthy structured representation required by Module 2. Module 2 produces security findings and verification conditions that constrain Module 3. Module 3 uses the same analysis pipeline again after transformation, creating a feedback loop in which the transformed program is not accepted merely because a new version of the code has been generated. The explanation layer connects the technical evidence to developer review.

3.5.2 Development and Design Process

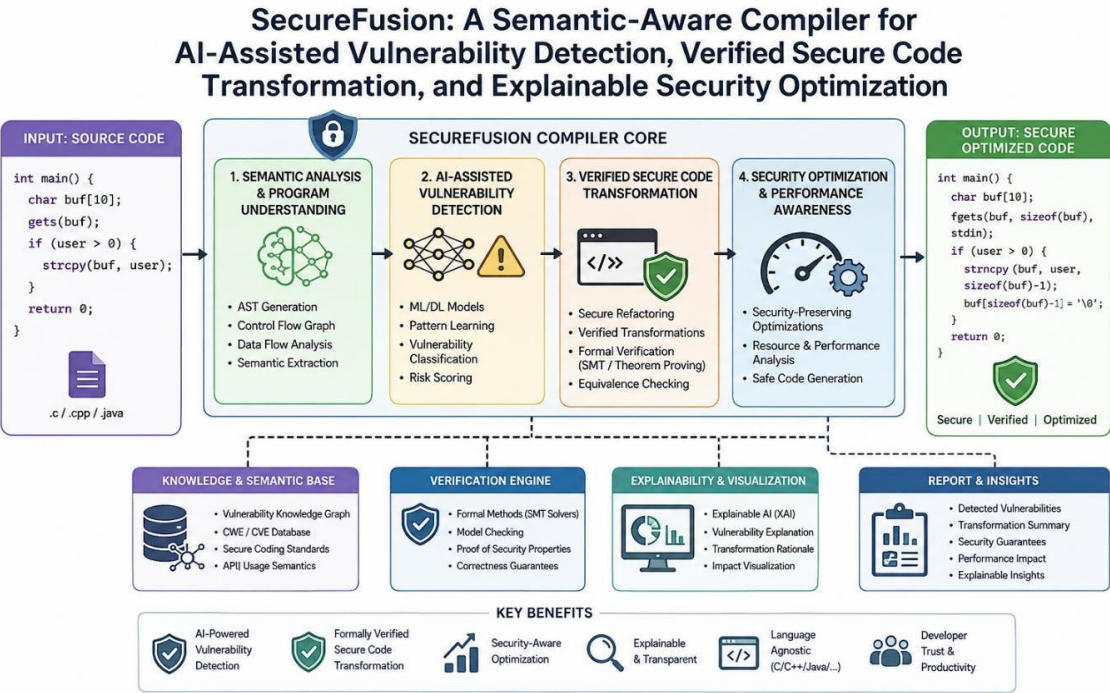


Figure 1. Development and Design Process

3.6 Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
AI in decision pipeline	AI as autonomous fixer	Fast and flexible	Risk of unverified changes	AI remains an assistant; deterministic verification remains mandatory.
Security analysis	Surface-level pattern matching only	Simple and fast	Contextual false positives/negatives	Use semantic/data-flow/control-flow context.

Transformation	Direct source replacement	Simple implementation	May change behaviour	Re-analyse and verify after transformation.
Explainability	Warning-only output	Minimal interface	Low developer trust/context	Report finding, change, expected effect and evidence.

3.7 Sustainability, Ethics, Safety, Risk & Security

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	Reduce unnecessary computation and repeated work where possible.	Efficient IR use, incremental analysis, reusable security rules and prioritization are identified as optimization directions.	Modular and reusable analysis is preferred over duplicated processing.
Ethics	AI output must not be treated as unquestionable code.	Need for traceability, verification, disclosure of AI-assisted resources and academic integrity.	AI recommendations are treated as candidate solutions only.
Health & Safety	Software-only project has no direct physical hazard pathway.	No hardware fabrication or hazardous physical process is part of the source project.	Focus remains on safe software behaviour and reliable verification.
Security	Automated remediation must not introduce new vulnerabilities.	Security re-analysis, semantic checks, secure coding guidance and regression tests.	Verification is placed before acceptance of a transformation.

3.7.1 Risk Register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
False positive vulnerability finding	Medium	Medium	Medium	Use semantic context, evidence and developer review.	Low–Medium
False negative vulnerability finding	Medium	High	High	Combine rules with semantic/data-flow analysis and benchmark cases.	Medium
AI-generated unsafe transformation	Medium	High	High	Re-run security and semantic verification; reject failed candidates.	Low–Medium
Functional regression after transformation	Medium	High	High	Regression tests and semantic-preservation checks.	Low–Medium
High computational cost	Medium	Medium	Medium	Incremental analysis, reusable rules and prioritization.	Medium
Poor explanation quality	Medium	Medium	Medium	Use structured evidence and before/after representations.	Low–Medium

CHAPTER 4 – IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

The development approach follows a modular engineering workflow. The source report organizes the system into source processing, semantic analysis, vulnerability detection, transformation, verification and explainability. This structure supports incremental development because each module can be reasoned about separately while the complete pipeline is tested as an integrated workflow.

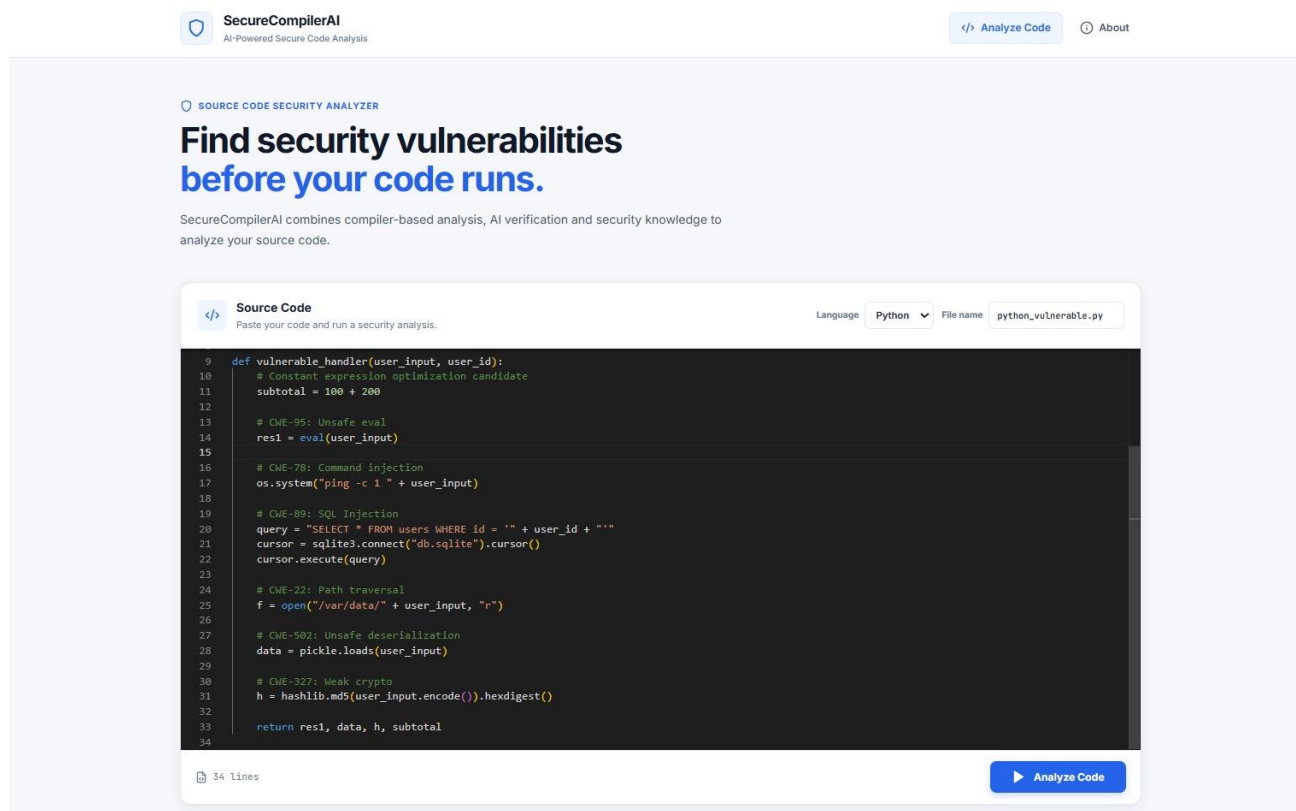
The project uses software-based resources rather than physical fabrication. The implementation concept relies on Python for prototype orchestration, parser technology such as ANTLR/PLY or equivalent, AST/IR structures, static-analysis components, AI/LLM-assisted reasoning, Git for version control, and automated testing frameworks.

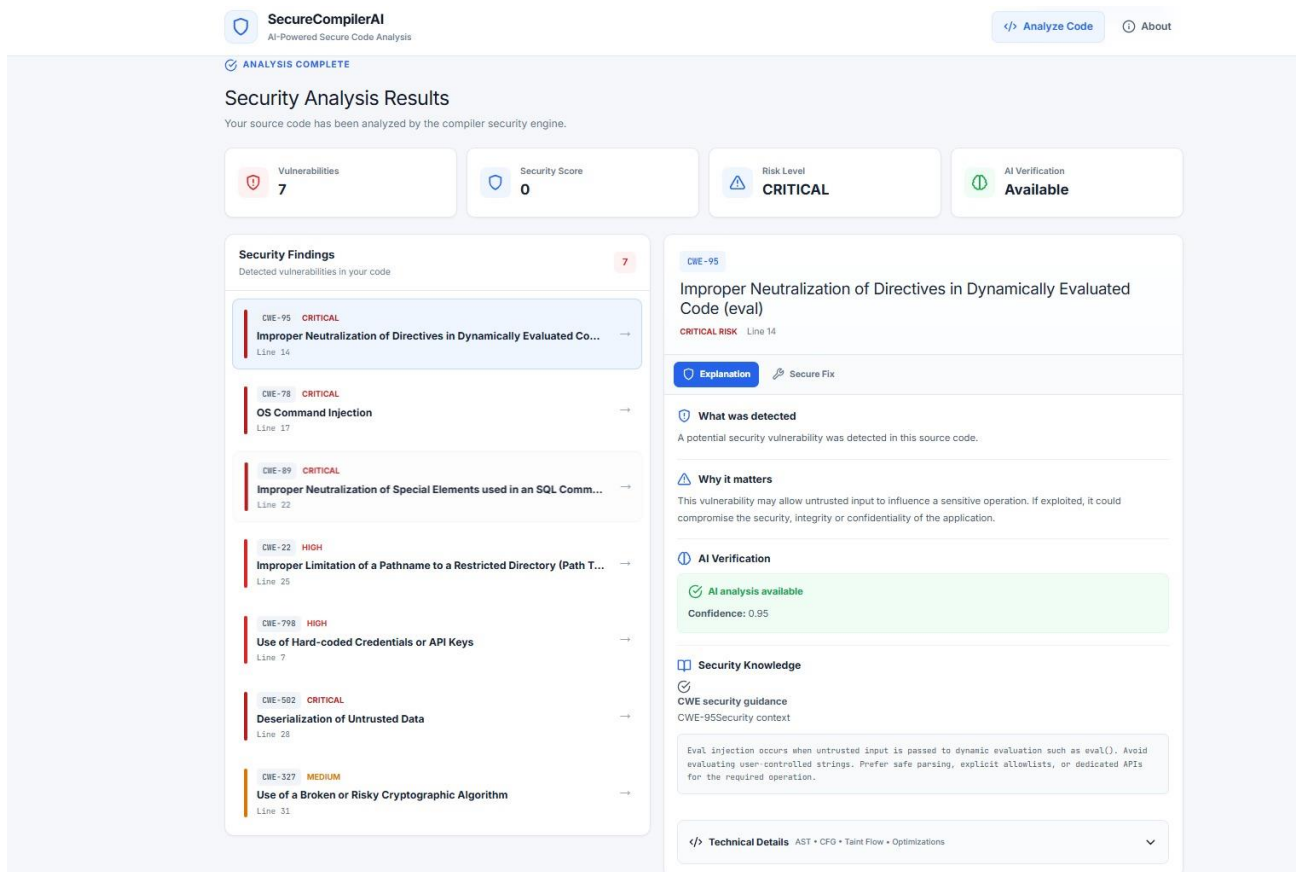
4.2 Software Implementation & Modern Tools Used

Tool / Software / Resource	Purpose	Stage Used
Python	Prototype implementation and orchestration	Implementation
ANTLR / PLY or equivalent parser technology	Lexical and syntactic analysis	Module 1
AST / Intermediate Representation	Structured representation for semantic reasoning	Modules 1–3
Static-analysis components	Data-flow and control-flow analysis	Module 2
Machine Learning / LLM-assisted component	Vulnerability explanation, prioritization and candidate recommendations	Modules 2–3
Git	Version control and change tracking	All stages
Automated testing frameworks	Unit, integration, regression and security test cases	Testing
Optional IDE / CI integration	Real-time or pipeline-based security feedback	Future / extension

4.2.1 Implementation Workflow

8. Tokenize and parse source code into structured representations.
9. Construct AST and relevant symbol/data-flow information.
10. Run security rules and semantic checks to identify suspicious behaviour.
11. Rank or explain findings using AI-assisted reasoning and contextual evidence.
12. Generate candidate secure transformations.
13. Re-analyse and verify transformed code against security and semantic constraints.
14. Produce an explainable report containing findings, changes and verification status.





4.3 Test Plan & Verification

The test plan is designed before results are interpreted. It covers syntax and parsing correctness, known vulnerable and secure code pairs, data-flow cases containing untrusted input and security-sensitive sinks, accepted transformation cases, negative cases where a proposed transformation must be rejected, and regression tests to ensure verified transformations do not break existing functionality.

Requirement / Specification	Test Method	Acceptance Criterion
Valid source parsing	Provide representative valid syntax inputs	Parser constructs the expected structured representation.
Invalid source handling	Provide invalid syntax inputs	Parser identifies structural/syntax failure without producing an invalid accepted program representation.
Vulnerability detection	Use representative vulnerable and secure code pairs	Security-relevant cases are distinguished with supporting evidence.
Source-to-sink reasoning	Use data-flow cases with untrusted sources and sensitive sinks	Relevant flow is identified and reported.
Transformation acceptance	Apply a supported secure	Candidate is accepted only

	transformation	after required checks pass.
Transformation rejection	Use a negative/unsafe candidate	Candidate is rejected when verification conditions fail.
Regression safety	Run existing functional tests after transformation	No unintended regression is observed for supported test cases.
Explanation quality	Review generated security report	Finding, change, expected effect and verification status are understandable.

4.4 Results

The source report describes the expected prototype outcome rather than providing a complete set of measured numerical benchmark values. Accordingly, this report does not fabricate accuracy, precision, recall, execution-time or cost figures. The results are documented as engineering outcomes that can be verified through the defined test plan.

Evaluation Area	Expected / Documented Outcome	Evidence Type
Parsing	Structured source representation is produced for supported syntax.	Parsing test cases / AST inspection
Semantic analysis	Security-relevant data/control relationships can be evaluated.	Data-flow and control-flow test cases
Vulnerability detection	Findings contain location, context and supporting evidence.	Security test cases
Transformation	Candidate fixes are re-analysed before acceptance.	Transformation verification cases
Regression	Existing functionality is checked after transformation.	Regression test suite
Explainability	Security report describes risk, change and verification status.	Generated explanation review

4.5 Data Analysis & Engineering Judgment

The main engineering judgment is that security improvement cannot be evaluated only by counting reduced warnings. A transformation that removes a warning but changes intended behaviour is not an acceptable secure transformation. Therefore, the evaluation must consider security findings together with semantic-preservation checks, regression behaviour, evidence quality and explanation usefulness.

The source identifies several important evaluation dimensions: vulnerability detection precision and recall on representative test cases, transformation correctness and semantic preservation, reduction in security findings after verified transformation, quality and usefulness of generated explanations, processing time, and scalability with increasing source-code size. Numerical values for these dimensions are not supplied in the source report and are therefore left for future measured evaluation.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective	Evidence / Engineering Activity	Achievement
Design semantic-aware compiler framework	Three-module architecture integrating compiler and security functions	Fully
Develop secure parsing and structured representations	Lexer/parser, AST/IR and symbol-table design	Fully
Identify vulnerabilities using semantic information	Data-flow/control-flow and security-aware semantic analysis	Fully at prototype-design level
Verify security before/after transformations	Re-analysis, semantic checks and verification conditions	Fully at framework level
Generate secure transformations	Candidate transformation workflow and supported transformation concepts	Fully at framework/prototype level
Provide explainable results	Risk, change, expected effect and verification status in output concept	Fully
Support extensibility	Modular separation of language, analysis, security and transformation rules	Fully

4.7 Strengths & Limitations

Strengths

- Integrates compiler structure with software-security reasoning.
- Uses semantic context rather than relying only on surface-level patterns.
- Treats AI output as a candidate that requires deterministic verification.
- Provides explainability as part of the security workflow.
- Supports modular extension to additional languages, security rules and analysis methods.

Limitations

- The documented project is a prototype architecture and is not presented as production-grade support for every programming language.
- Complete formal verification of all possible program properties is outside the stated scope.
- Security detection can still face false positives and false negatives.
- Repeated analysis before and after transformation can increase computational cost.
- The source report does not provide a full numerical benchmark dataset, measured cost table, or completed Gantt chart.

4.8 Project Planning, Roles & Budget

Stage	Main Activities	Primary Responsibility
Stage 1	Compiler/security concept review and problem definition	All members
Stage 2	Module 1 design and structured representation	Pooja Sree M
Stage 3	Module 2 semantic security verification	Sofia R V
Stage 4	Module 3 transformation and explainability	Prartana T
Stage 5	Integration, testing and regression analysis	All members
Stage 6	Documentation, reflection and final report	All members

Budget note: the source report does not provide a project cost breakdown. Since the documented solution is software-oriented and does not specify purchased hardware or fabrication, no unsupported monetary figures are introduced in this report. A final budget can be completed from actual receipts, subscriptions, hosting charges, or other project expenses if such records are available.

CHAPTER 5 – CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

Secure CompilerAI presents a semantic-aware compiler architecture for AI-assisted vulnerability detection, verified secure code transformation, and explainable security optimization. The project extends the traditional compiler pipeline by integrating security intelligence into source-code analysis and semantic verification rather than treating security as an isolated post-compilation activity.

The three modules provide a clear progression: Module 1 constructs secure and trustworthy program representations; Module 2 evaluates those representations using semantic security properties; and Module 3 generates, verifies, and explains security-preserving transformations. This separation of responsibilities improves modularity while maintaining a connected analysis workflow.

The central principle is verification before acceptance. AI recommendations are treated as candidate solutions rather than authoritative code. By combining AI assistance with compiler analysis and verification, the framework aims to make automated security optimization more trustworthy, explainable and suitable for developer workflows.

5.2 Future Work

- Support additional programming languages through reusable grammars and intermediate representations.
- Add richer taint analysis, symbolic execution and inter-procedural data-flow analysis.
- Improve AI-assisted ranking using project-specific security context.
- Introduce stronger formal or proof-oriented verification for selected transformation classes.
- Provide IDE integration with real-time diagnostics and transformation previews.
- Build a benchmark suite containing vulnerable, patched and semantically equivalent programs.
- Improve explanation quality using structured evidence such as source-to-sink paths and before/after IR.
- Integrate the framework with development and CI/CD workflows after sufficient validation.

5.3 Professional Learning & Individual Reflection

5.3.1 New Knowledge Acquired

- Compiler phases can be extended beyond syntax and performance to include security properties and trustworthy transformation.
- AST/IR processing, data-flow reasoning and control-flow analysis are useful for security-aware program understanding.
- AI-assisted programming requires systematic validation rather than blind acceptance of generated output.

- Semantic equivalence and regression checking are important when source code is automatically transformed.

5.3.2 Engineering & Professional Skills Developed

- Python programming and prototype orchestration.
- Parser construction and structured program representation.
- Static analysis, data-flow and control-flow reasoning.
- Secure coding and vulnerability analysis.
- Unit, integration, security and regression testing.
- Version control, traceability and technical documentation.
- Trade-off analysis between security, correctness, performance and usability.

5.3.3 Individual Reflection – Pooja Sree M

The main engineering challenge addressed through the assigned work was maintaining a trustworthy transition from source text to structured program representation. Secure parsing was treated as a foundation for later semantic security analysis, because incorrect or incomplete structure would affect every downstream module. A key decision was to preserve compiler-oriented representations such as AST and symbol information rather than relying only on text patterns. This work strengthened understanding of lexical analysis, parsing, structured representations and secure software reasoning.

5.3.4 Individual Reflection – Sofia R V

The main challenge in the semantic verification role was reasoning about security properties using program context rather than isolated syntax. The project therefore emphasizes control-flow and data-flow relationships, source-to-sink tracking and verification conditions. A key engineering decision was to use AI-assisted reasoning for prioritization or explanation without allowing it to bypass deterministic checks. This improved understanding of static analysis, vulnerability classification and the relationship between semantic information and security decisions.

5.3.5 Individual Reflection – Prartana T

The main challenge in the transformation role was ensuring that a candidate security fix is not accepted simply because it appears to remove a warning. The selected workflow applies the transformation and then re-runs semantic and security checks before acceptance. A key decision was therefore to treat secure transformation as a verified process and to produce an explanation containing the original risk, changed code, expected effect and verification evidence. This strengthened understanding of program transformation, optimization and trustworthy AI-assisted development.

REFERENCES

- [1] A. V. Aho, M. S. Lam, R. Sethi and J. D. Ullman, Compilers: Principles, Techniques, and Tools, 2nd ed., Pearson.
- [2] A. W. Appel, Modern Compiler Implementation, Cambridge University Press.
- [3] A. V. Aho, R. Sethi and J. D. Ullman, Principles of Compiler Design, Addison-Wesley.
- [4] ISO/IEC 25010, Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model.
- [5] OWASP Foundation, OWASP Top 10: The Ten Most Critical Web Application Security Risks.
- [6] MITRE, Common Weakness Enumeration (CWE), software weakness classification and security reference.
- [7] MITRE, Common Vulnerabilities and Exposures (CVE), vulnerability identification reference.
- [8] V. D'Silva, D. Kroening and R. Kurshan, research on software verification and program analysis.
- [9] M. Fowler, Refactoring: Improving the Design of Existing Code, Addison-Wesley.
- [10] C. Lattner et al., LLVM Compiler Infrastructure documentation and research resources.
- [11] ANTLR Project, ANTLR parser generator documentation and reference material.
- [12] Python Software Foundation, Python Language Reference and Abstract Syntax Tree documentation.

APPENDICES

The appendices provide supporting engineering material without replacing the main explanation in Chapters 1–5. Where the source report does not contain measured numerical records, this document provides structured calculation methods, evidence templates, diagrams, test formats, and documentation guidance rather than fabricated results.

APPENDIX A – DETAILED CALCULATIONS

This appendix records the principal analytical relationships relevant to the proposed framework. The source project is software-oriented; therefore, calculations are primarily logical, complexity-oriented, and verification-oriented rather than hardware sizing calculations.

A.1 Verification Decision Logic

A candidate transformation can be represented conceptually as an accepted change only when the required conditions are satisfied:

Acceptance = Security_Check AND Semantic_Preservation_Check AND Regression_Check

If any mandatory condition fails, the candidate should be rejected or returned for further analysis. This reflects the project principle that an AI-generated recommendation is a candidate solution and not automatically trusted code.

A.2 Vulnerability Evidence Structure

Field	Purpose
Vulnerability type	Identifies the security weakness being reported.
Location	Identifies the affected source region or program representation.
Source	Identifies the potentially untrusted input or originating operation.
Sink	Identifies the security-sensitive operation or destination.
Flow evidence	Shows the relevant data/control relationship.
Severity / priority	Helps order findings for developer attention.
Recommendation	Describes the candidate secure transformation.
Verification status	Records whether the candidate passed required checks.

A.3 Complexity Considerations

Let N represent the size of the source representation and E represent the number of relevant control/data-flow relationships. Lexing and parsing are generally organized around the source representation, while graph-based analyses depend on nodes and edges in the constructed representations. Repeated re-analysis after transformation increases the overall processing work because the pipeline is executed again for the modified program.

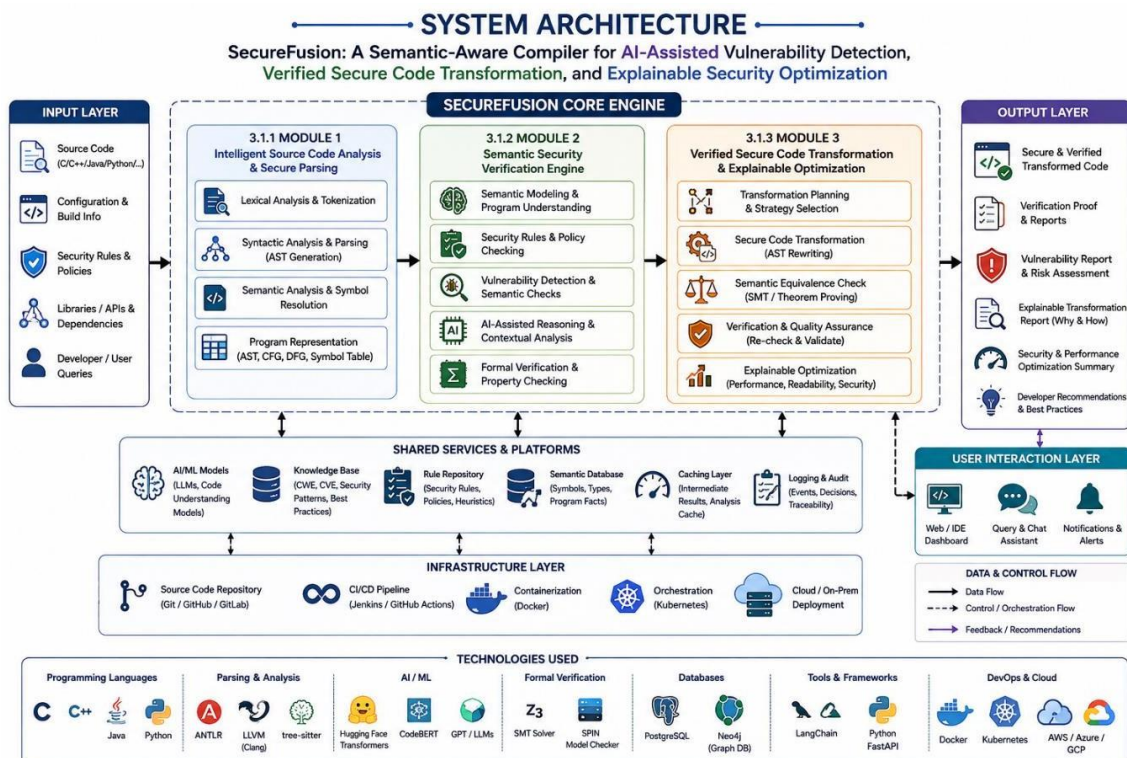
For this reason, the source report identifies efficient intermediate representations, incremental analysis, reusable security rules and prioritization of high-risk findings as optimization directions.

A.4 Transformation Verification Checklist

- Candidate transformation corresponds to the detected vulnerability.
- Required security property is explicitly identified.
- Program representation remains structurally valid.
- Relevant semantic/security analysis is repeated after transformation.
- Semantic-preservation conditions are satisfied.
- Regression tests do not show unintended functional breakage.
- Explanation records the change and verification evidence.

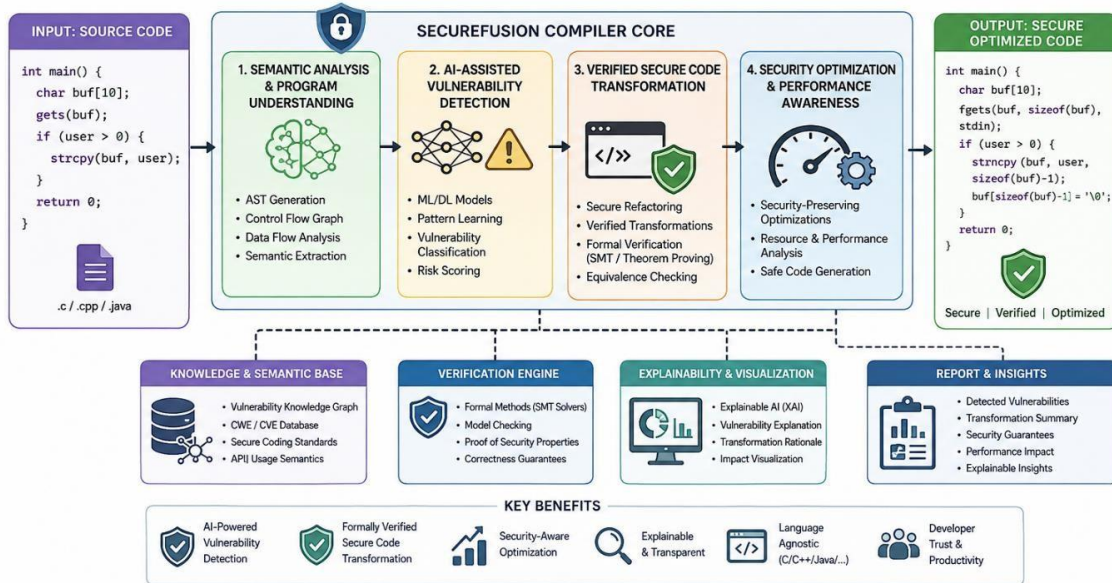
APPENDIX B – ENGINEERING DRAWINGS / SCHEMATICS

The following figures are reproduced from the project material and placed here as supporting engineering schematics.



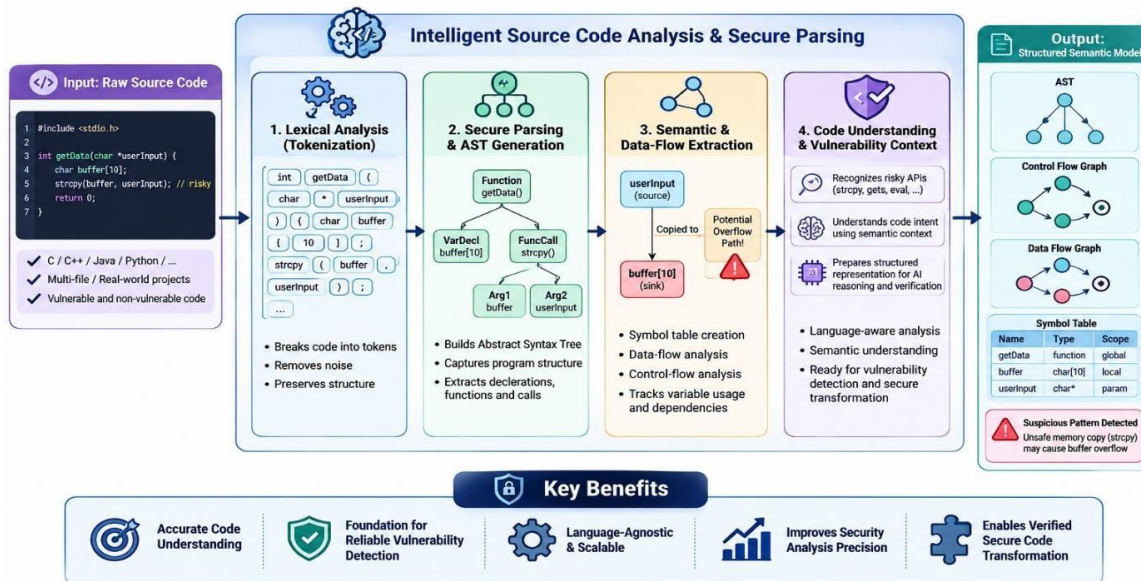
Appendix B Figure 1. Secure CompilerAI System Architecture

SecureFusion: A Semantic-Aware Compiler for AI-Assisted Vulnerability Detection, Verified Secure Code Transformation, and Explainable Security Optimization



Appendix B Figure 2. Development and Design Process

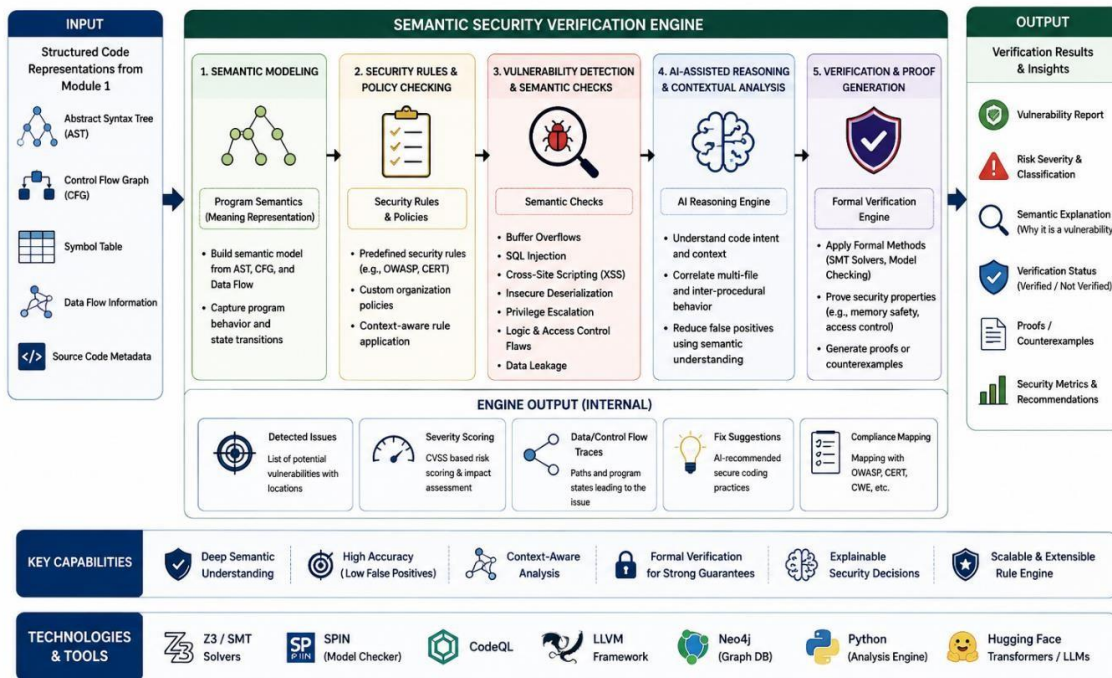
3.1.1 Module 1: Intelligent Source Code Analysis & Secure Parsing "Understand Code Semantics Before Securing It"



Appendix B Figure 3. Module 1 – Secure Parsing

3.1.2 Module 2: Semantic Security Verification Engine

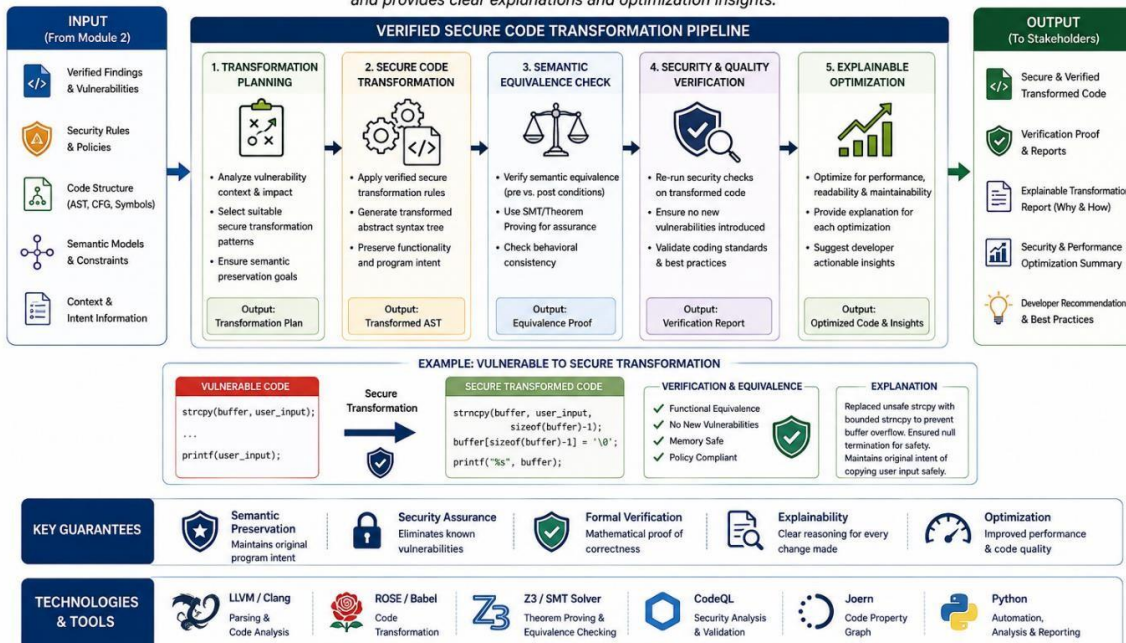
Verifies code semantics, detects vulnerabilities, and ensures security properties using formal methods and AI-assisted reasoning.



Appendix B Figure 4. Module 2 – Semantic Verification

3.1.3 Module 3: Verified Secure Code Transformation & Explainable Optimization

Transforms vulnerable code into secure, semantically equivalent code with formal verification and provides clear explanations and optimization insights.



Appendix B Figure 5. Module 3 – Verified Transformation

APPENDIX C – SOURCE CODE / CODE REPOSITORY INFORMATION

The source report identifies Python as the prototype implementation/orchestration language and ANTLR/PLY or equivalent parser technology for lexical and syntactic analysis. It also identifies AST/IR processing, static-analysis components, AI/LLM-assisted components, Git, and automated testing frameworks as relevant implementation resources.

C.1 Suggested Repository Structure

A maintainable repository can separate the three modules and shared resources so that each engineering responsibility remains traceable.

Folder / Component	Purpose
src/	Main implementation modules.
src/parser/	Lexer, parser and secure parsing logic.
src/analysis/	AST/IR, control-flow and data-flow analysis.
src/security/	Security rules and vulnerability finding logic.
src/transform/	Candidate transformation and verification workflow.
src/explain/	Explainable security and optimization reporting.
tests/unit/	Module-level tests.
tests/integration/	End-to-end pipeline tests.
tests/regression/	Regression tests for transformations.
docs/	Architecture, design and testing documentation.

C.2 Essential Pseudocode

1. Read source program.
2. Tokenize input.
3. Parse tokens and construct AST/IR.
4. Build symbol, control-flow and data-flow information.
5. Run security rules and record findings.
6. Use AI assistance to prioritize/explain findings where applicable.
7. Generate a candidate secure transformation.
8. Apply candidate transformation.
9. Re-run semantic and security analysis.
10. Check semantic-preservation and regression conditions.
11. Accept verified candidate; otherwise reject.

12. Produce an explainable report.

C.3 Version-Control Evidence

- Use meaningful commit messages describing module-level changes.
- Keep transformation-rule changes traceable to test cases.
- Maintain regression tests when modifying analysis or transformation logic.
- Record major architecture decisions in project documentation.

APPENDIX D – DATASHEETS / SOFTWARE RESOURCE SHEETS

Because the project is software-oriented, conventional hardware datasheets are not applicable. The equivalent supporting material is a software-resource sheet describing the tools and their engineering purpose.

Resource	Role in Project	Evidence to Maintain
Python	Prototype implementation and orchestration	Environment/version record and source files
ANTLR / PLY / equivalent	Lexical and syntactic processing	Grammar/parser files and test inputs
AST / IR	Program representation	AST/IR examples and transformation records
Static analysis	Data-flow/control-flow reasoning	Analysis rules and test cases
AI / LLM-assisted component	Explanation, prioritization and candidate recommendations	Prompts/outputs where institutionally appropriate and verification evidence
Git	Version control	Commit history and repository records
Testing framework	Automated verification	Test suite and execution records

APPENDIX E – EXPERIMENTAL / RAW DATA

The source report defines the categories of testing and evaluation but does not provide a completed numerical raw-data table. To avoid fabricating results, this appendix provides a ready-to-use raw-data structure for recording actual test execution.

E.1 Evaluation Metrics to Record

- Precision and recall for representative vulnerability-detection cases.
- Number of correctly accepted transformations.
- Number of correctly rejected unsafe transformations.
- Regression count after accepted transformations.
- Reduction in security findings after verified transformation.
- Processing time for increasing source-code sizes.
- Qualitative usefulness of generated explanations.

Important: the project report should be updated with measured values only after actual execution of the corresponding test cases.

APPENDIX F – ETHICS / INSTITUTIONAL APPROVAL

The project is a software/compiler-security study and does not involve physical experimentation with human subjects, animals, hazardous chemicals, or fabrication according to the documented project scope. The primary ethical requirements therefore concern academic integrity, responsible AI use, software-security responsibility, data provenance, and transparent reporting.

- Do not present AI-generated output as verified merely because it was generated by an AI system.
- Disclose AI-assisted resources when required by institutional policy.
- Do not fabricate test results, benchmark values, screenshots, or experimental observations.
- Maintain traceability between reported findings, test inputs, transformations and verification outcomes.
- Respect software licenses and intellectual-property requirements for external code, datasets and open-source resources.
- Retain human review for security-sensitive decisions.

APPENDIX G – PROJECT MEETING / PROGRESS RECORDS

The source report does not contain dated meeting minutes. The following structured record is provided so the team can enter actual progress evidence without inventing dates or discussions.

APPENDIX H – USER / INDUSTRY FEEDBACK

No completed user or industry feedback dataset is included in the source report. This appendix provides a feedback structure aligned with the project’s stated stakeholders.

APPENDIX I – PUBLICATIONS / PATENTS / COMPETITIONS / PRODUCT OUTCOMES

The source report does not identify a completed publication, patent, competition award, or commercial product outcome. The project outcome documented in the source is a compiler-security framework/prototype concept with modular implementation and evaluation requirements.

Outcome Category	Status in Current Report	Evidence / Record
Publication	Not specified in source report	Add manuscript/publication details if completed.
Patent	Not specified in source report	Add patent/application details if applicable.
Competition	Not specified in source report	Add participation/award details if applicable.
Product / Prototype	Prototype framework described	Architecture, modules and implementation approach.
Repository	Git identified as version-control resource	Insert approved repository information if available.

APPENDIX J – INDIVIDUAL CONTRIBUTION EVIDENCE

This appendix provides a traceability matrix for the team-declared individual contributions. The percentages are the team-declared allocation used in this report; supporting evidence should be attached where available.

Student	Module / Responsibility	Development Evidence	Testing Evidence	Report Evidence	Contribution
Pooja Sree M	Module 1 – Secure Parsing	Parser/AST/symbol representation work	Parsing and structural test cases	Ch. 1, Module 1, methodology/testing	35%
Sofia R V	Module 2 – Semantic Verification	Data/control-flow and security analysis	Vulnerability and negative cases	Ch. 2, Module 2, verification/testing	33%
Prartana T	Module 3 – Verified Transformation	Transformation, verification and explanation	Transformation/regression tests	Module 3, architecture, conclusion/future work	32%

J.1 Evidence Checklist

- Screenshots of module implementation or development environment.
- Relevant source-code commits or repository history.
- Test-case records associated with each module.
- Architecture/design diagrams prepared or maintained by the team.
- Sections of the report drafted or reviewed by each member.
- Presentation/viva material demonstrating individual understanding.

CAPSTONE PROJECT OUTCOME MAPPING SHEET

The following mapping follows the outcome categories provided in the capstone template. Final assessment is to be completed by the department/faculty assessor.

Outcome Evidence	SO / Area	Location in This Report
Complex engineering problem formulation	SO1	Chapter 1 and Chapter 2
Engineering design under constraints	SO2	Chapter 3
Written/oral technical communication	SO3	Whole report and Viva
Ethics and professional responsibility	SO4	Chapter 3.7 and Appendix F
Teamwork and leadership	SO5	Chapter 4.8 and Appendix J
Experimentation, analysis, engineering judgment	SO6	Chapter 4.3–4.6 and Appendix E
Independent acquisition of new knowledge	SO7	Chapter 5.3
Standards	SO2	Chapter 3.2
Sustainability and societal/environmental impact	SO2/SO4	Chapter 3.7
Risk assessment and mitigation	SO2/SO4	Chapter 3.7.1
Security considerations	Relevant SO	Chapter 3.7 and throughout
Integrated/system-level engineering	SO1/SO2	Chapter 3.5
Project planning and management	SO5	Chapter 4.8 and Appendix G
Individual contribution	SO1–SO7	Contribution Statement and Appendix J